```
import React, { useEffect, useState, useCallback } from "react";
import {
  SafeAreaView,
  View,
  Text,
  Switch,
  StyleSheet,
  StatusBar,
  Platform,
} from "react-native";

// ---------- 1. YOUR SETTINGS (change these) ----------
// Get a NEW token from Blynk (Device > Device Info). Never push a real
token to GitHub.
const BLYNK_TOKEN = "YourAuthToken";
// Your Blynk server. India is usually "blr1.blynk.cloud". Others:
"blynk.cloud", "sgp1.blynk.cloud".
const BLYNK_SERVER = "blr1.blynk.cloud";
// Voltage above this lets the relay switch on (same as vin >= 2 in your
ESP32 code)
const MIN_VOLTAGE = 2;
// How often the app refreshes (milliseconds)
const REFRESH_MS = 2000;
// -----------------------------------------------------

const BASE = `https://${BLYNK_SERVER}/external/api`;

export default function App() {
  const [voltage, setVoltage] = useState(null);
  const [deviceOnline, setDeviceOnline] = useState(false);
  const [loadEnabled, setLoadEnabled] = useState(true);
  const [error, setError] = useState("");

  // Read voltage (V0), device status, and the switch (V1)
  const refresh = useCallback(async () => {
    try {
      const [v, online, sw] = await Promise.all([
        fetch(`${BASE}/get?token=${BLYNK_TOKEN}&V0`).then((r) =>
r.text()),
        fetch(`${BASE}/isHardwareConnected?token=${BLYNK_TOKEN}`).then((r)
```

```
=>
          r.text()
        ),
        fetch(`${BASE}/get?token=${BLYNK_TOKEN}&V1`).then((r) =>
r.text()),
      ]);
      const num = parseFloat(v);
      if (isNaN(num)) throw new Error("Bad token or server");
      setVoltage(num);
      setDeviceOnline(online.trim() === "true");
      // In your ESP32 code the relay can turn on only when V1 == 0
      setLoadEnabled(sw.trim() === "0");
      setError("");
    } catch (e) {
      setError("Can't reach Blynk. Check your token, server and
internet.");
    }
  }, []);

  useEffect(() => {
    refresh();
    const timer = setInterval(refresh, REFRESH_MS);
    return () => clearInterval(timer);
  }, [refresh]);

  // Send the switch value to V1 (0 = load allowed, 1 = load forced off)
  const toggleLoad = async (enabled) => {
    setLoadEnabled(enabled);
    try {
      await fetch(
        `${BASE}/update?token=${BLYNK_TOKEN}&V1=${enabled ? 0 : 1}`
      );
    } catch (e) {
      setError("Couldn't change the load. Try again.");
      setLoadEnabled(!enabled);
    }
  };

  const sunnyEnough = voltage !== null && voltage >= MIN_VOLTAGE;
  const loadOn = sunnyEnough && loadEnabled;

  return (
    <SafeAreaView style={s.screen}>
```

```
      <StatusBar barStyle="dark-content" />

      <Text style={s.title}>SunAid</Text>
      <Text style={s.subtitle}>Your solar panel, in your pocket</Text>

      {/* Voltage */}
      <View style={s.hero}>
        <Text style={s.heroLabel}>Panel voltage</Text>
        <Text style={s.heroValue}>
          {voltage === null ? "--" : voltage.toFixed(1)}
          <Text style={s.heroUnit}> V</Text>
        </Text>
        <Text style={s.heroNote}>
          {voltage === null
            ? "Waiting for data..."
            : sunnyEnough
            ? "Good sunlight. Power is available."
            : "Low sunlight. The load stays off."}
        </Text>
      </View>

      {/* Load control */}
      <View style={s.row}>
        <View style={{ flex: 1 }}>
          <Text style={s.rowTitle}>Load (relay)</Text>
          <Text style={s.rowSub}>
            {loadEnabled
              ? "On when the sun is strong enough"
              : "Forced off from the app"}
          </Text>
        </View>
        <Switch
          value={loadEnabled}
          onValueChange={toggleLoad}
          trackColor={{ true: "#E8A317", false: "#C9C3B5" }}
          thumbColor="#FFFFFF"
        />
      </View>

      {/* Status */}
      <View style={s.row}>
        <Text style={s.rowTitle}>Device</Text>
        <Text style={[s.badge, deviceOnline ? s.good : s.bad]}>
```

```
          {deviceOnline ? "Online" : "Offline"}
        </Text>
      </View>
      <View style={s.row}>
        <Text style={s.rowTitle}>Load right now</Text>
        <Text style={[s.badge, loadOn ? s.good : s.neutral]}>
          {loadOn ? "Running" : "Off"}
        </Text>
      </View>

      {error !== "" && <Text style={s.error}>{error}</Text>}
    </SafeAreaView>
  );
}

const s = StyleSheet.create({
  screen: {
    flex: 1,
    backgroundColor: "#F6F1E4",
    paddingHorizontal: 22,
    paddingTop: Platform.OS === "android" ? 48 : 16,
  },
  title: { fontSize: 34, fontWeight: "800", color: "#12403C" },
  subtitle: { fontSize: 15, color: "#5B6B66", marginBottom: 22 },
  hero: {
    backgroundColor: "#12403C",
    borderRadius: 28,
    padding: 26,
    marginBottom: 18,
  },
  heroLabel: { color: "#BFD8D2", fontSize: 15 },
  heroValue: { color: "#FFC94D", fontSize: 72, fontWeight: "800" },
  heroUnit: { fontSize: 28, color: "#FFE3A0" },
  heroNote: { color: "#E7F1EE", fontSize: 15, marginTop: 4 },
  row: {
    backgroundColor: "#FFFFFF",
    borderRadius: 16,
    padding: 16,
    marginBottom: 12,
    flexDirection: "row",
    alignItems: "center",
    justifyContent: "space-between",
  },
```

```
  rowTitle: { fontSize: 17, fontWeight: "600", color: "#12403C" },
  rowSub: { fontSize: 13, color: "#6B7A75", marginTop: 2 },
  badge: {
    paddingHorizontal: 14,
    paddingVertical: 6,
    borderRadius: 20,
    fontWeight: "700",
    overflow: "hidden",
  },
  good: { backgroundColor: "#DDF3E4", color: "#1B7A3A" },
  bad: { backgroundColor: "#FBE0DC", color: "#B3261E" },
  neutral: { backgroundColor: "#ECE7D8", color: "#5B6B66" },
  error: { color: "#B3261E", marginTop: 8, fontSize: 14 },
});
```